Transfer Learning-Enhanced Explainable Graph Neural Networks for Memory-Only Ransomware Detection

Avanish Kumar
Dept. of Information Technology and Computer Application
Madan Mohan Malaviya University of Technology
Gorakhpur, India
callavikumar@gmail.com

Rohan Kushwaha
Dept. of Information Technology and Computer Application
Madan Mohan Malaviya University of Technology
Gorakhpur, India
kushwaharohan2003@gmail.com

Prachi Verma
Dept. of Information Technology and Computer Application
Madan Mohan Malaviya University of Technology
Gorakhpur, India
pvitca@mmmut.ac.in

***Abstract*— Memory-only ransomware runs entirely in RAM, evading traditional signature-based and file-scanning defenses. This paper presents a framework using Graph Neural Networks (GNNs), transfer learning, and explainable AI (XAI) to detect such threats. We construct fully connected feature graphs from memory dump attributes (nodes = system features, edges = pairwise interactions) as a baseline representation; semantic graphs are left for future work. A two-layer Graph Convolutional Network (GCN) achieves 93.22% accuracy and 0.9309 F1-score on the CIC-MalMem-2022 dataset. To improve generalization to unseen ransomware families, we apply cross-family transfer learning (pre-training on known families, fine-tuning on a target family), raising accuracy to 94.27% with 97.55% precision – outperforming the pre-trained model and traditional ML baselines. GNNExplainer provides interpretable explanations, identifying key indicators such as ldrmodules.not_in_load and pslist.avg_threads. The results show that even simple graph constructions, when combined with transfer learning and XAI, offer effective and transparent memory-resident ransomware detection.**

Keywords— Ransomware Detection, Transfer Learning, Explainable Graph Neural Network, Memory-resident

I. Introduction

A. Background and Motivation

Memory-only ransomware executes entirely in RAM, never writing malicious code to disk. This behavior evades traditional signature-based antivirus and static file scanners, which rely on disk artefacts [1], [8]. As ransomware families grow more sophisticated, new detection methods must operate directly on volatile memory and adapt to unseen variants without requiring labelled samples for every new family.

B. Limitations of Existing Approaches

Existing malware detection techniques suffer from several gaps. Signature-based methods fail against zero-day and polymorphic ransomware [10]. Static analysis is ineffective for memory-only threats [8]. Traditional machine learning (e.g., Random Forest, XGBoost) on hand-crafted features lacks the capacity to model complex feature interactions [13], [15]. Deep learning methods such as CNNs and RNNs treat inputs as flat sequences, ignoring relational structure. Graph Neural Networks (GNNs) have shown promise [3], [5], but prior GNN works either do not target memory-resident ransomware, lack transfer learning to generalize across families, or provide no explainability [1], [9].

C. Proposed Solution

We propose TL-XGNN, a framework that combines Graph Neural Networks, cross-family transfer learning, and explainable AI for memory-only ransomware detection. We construct fully connected feature graphs from memory dump attributes (55 features, each a node; all pairwise edges) to enable GNN message passing. A two-layer Graph Convolutional Network (GCN) serves as the base detector. To improve generalization to unseen ransomware families, we apply cross-family transfer learning: pre-train on all known ransomware families (Ako, Shade, Conti, Pysa, etc.) and benign samples, then fine-tune on a small set of a target family (Maze). Finally, we integrate GNNExplainer to provide interpretable explanations of model decisions.

D. Experimental Validation

We evaluate on the CIC-MalMem-2022 dataset (58,058 memory dumps). The baseline GCN achieves 93.22% accuracy and 0.9309 F1-score. Cross-family transfer learning raises accuracy to 94.27% with 97.55% precision, outperforming pre-trained and traditional ML baselines. GNNExplainer identifies key features (e.g., ldrmodules.not_in_load, pslist.avg_threads) as top indicators. The paper is organised as follows: Section II reviews related work; Section III details the methodology; Section IV describes the experimental setup; Section V presents results; Section VI concludes.

II. Related Works

Previous research will be summarized based on three important aspects: graph neural networks for malware detection; transfer learning for addressing newly emerging threats; and explainable AI for Cybersecurity. Lastly, we highlight the research gap that necessitates the proposed framework.

A. Graph Neural Networks for Malware Detection

Several studies have applied GNNs to malware detection using diverse graph representations. Paper [1] used a GNN with self-supervised learning on binary-as-graph representations. DawnGNN [3] converted API call sequences into graphs and augmented node features with BERT embeddings from official Windows documentation. MDGraph [8] recursively disassembled memory dumps to build function call graphs and applied a GNN, making it the closest to our memory-only setting. Paper [5] employed an

attention-gated GNN on audit logs, while Paper [9] combined GCN with attention and feature selection for resource-constrained IIoT environments. However, none of these works simultaneously incorporate transfer learning and explainable AI for memory-resident ransomware.

B. Transfer Learning for Malware Detection

Transfer learning has been explored to address concept drift and data scarcity. Paper [17] systematically evaluated transfer learning for temporal concept drift. Paper [18] combined knowledge transfer with ensemble methods for malware variant detection on the Microsoft dataset, achieving moderate recall. Generative approaches such as tDCGAN [20] used a deep autoencoder to stabilize DCGAN training and transferred the discriminator's knowledge to a detector, reaching 95.74% accuracy for zero-day attacks. Nevertheless, transfer learning has not been integrated with GNNs for memory-only ransomware detection – a gap our work fills.

C. Explainable AI (XAI) for Malware and Ransomware

Interpretability is critical for security analysts. Surveys [10], [11] provide comprehensive overviews of XAI techniques in malware analysis. XRGuard [12] used SHAP and LIME on ETW logs for ransomware detection, achieving 99.69% accuracy. Paper [13] applied XAI to tree-based models on CIC-MalDroid, reaching 94% F1. Paper [14] employed inherently explainable graphical models (Markov Random Fields, Bayesian Networks) on hardware performance counters. Among GNN works, only Paper [4] explicitly integrates SHAP for APT attribution. Our framework adds GNNExplainer to provide node-level explanations specifically for memory-resident ransomware.

D. Memory-Based Malware Detection

Only a few studies analyze memory dumps directly. MDGraph [8] is the most relevant, using memory dumps and GNNs on packed malware but lacking transfer learning and explainability. The CIC-MalMem-2022 dataset [9] was designed for memory-obfuscated malware evaluation, but prior GNN evaluations on it focused on resource efficiency for IIoT, not on ransomware-specific transfer learning or XAI. Our work is the first to propose a transfer learning-enhanced, explainable GNN tailored to memory-only ransomware detection.

Gap: Memory-only ransomware detection remains underexplored. Our work is the first to propose a transfer learning-enhanced, explainable GNN tailored to this scenario.

TABLE I
COMPARISON OF MALWARE DETECTION APPROACHES

Authors	Detection Technique	Dataset & Features	Best Accuracy	Limitations
Pengbin Feng et al. [3]	GAT + BERT (DawnGNN)	Windows API calls + official API documentation	Not specified	No memory dumps; no transfer learning; no XAI
Qiyuan Li et al. [8]	GNN + doc2vec (MDGraph)	Memory dumps → function call graphs	Not specified	No transfer learning; no XAI
M. Adnan et al. [12]	XRGuard (ETW logs + SHAP/LIME)	ETW logs (file I/O patterns)	99.69%	Not graph-based; no memory dumps; no transfer learning
Kim et al. [20]	tDCGAN (DAE + DCGAN + transfer learning)	Not specified (malware binaries)	95.74%	No GNN; no memory dumps; no XAI

III. METHODOLOGY

This section presents the designed framework of memory-based ransomware detection: TL-XGNN. The framework consists of four main parts as shown in Figure 1: (i) graph construction from memory dump feature representation, (ii) GCN backbone, (iii) cross-family transfer learning and (iv) explainability by GNNExplainer.

A. Graph Construction from Memory Dumps

The dataset employed in this paper is the CIC-MalMem-2022 [9], which consists of 58,058 memory dump examples where each example is described by 55 feature values produced by Volatility (such as pslist.avg_threads, handles.nfile, ldrmodules.not_in_load). The labels assigned to each data instance can be Benign (0) or Malware (1), because the dataset provides only extracted features (not raw memory), we construct a fully connected feature graph for each sample. Let a sample be a feature vector $f \in R55$.We create a graph G=(V,E) where:

- Nodes: V={v1,v2,…..,v55}, each node corresponds to one feature.
- Node features: For node vi, its feature is the scalar value fi, after standardization). Thus xi = [fi].
- Edges: E is the set of all ordered pairs (vi,vj) for i,j= 1,….,55, including self-loops. This results in a complete directed graph with 55×55=3025 edges.

This baseline graph enables message passing between all features, allowing the GNN to learn feature interactions. All feature values are standardized to zero mean and unit variance.

B. Graph Convolutional Network Architecture

We employ a simple yet effective GCN model, denoted as $f\theta$. It consists of:

- Input layer: Node features $xi \in R1$ (after graph construction).

- First GCN layer:

$h^{(1)} = \mathrm{ReLU}\left(\widetilde{D^{-\frac{1}{2}}AD^{-\frac{1}{2}}}XW^{(1)}\right)$, where $\widetilde{A} = A + I$ (adjacency with self-loops), $\widetilde{D}$ is the degree matrix of \widetilde{A}, W(1) ∈ R1x16. Output dimension: 16.

- Second GCN layer:

$h^{(2)} = \mathrm{ReLU}\left(\widetilde{D^{-\frac{1}{2}}AD^{-\frac{1}{2}}}h^{(1)}W^{(2)}\right)$, with $W^{(2)} \in$ R16x16. Output dimension: 16

- Global mean pooling:

$$h_{graph} = \frac{1}{|V|}\sum_{i \in V} h_i^{(2)}$$

- Linear classifier:

$$\hat{y} = \mathrm{softmax}\left(h_{graph}W_{out} + b\right), \quad W_{out} \in R^{16 \times 2}$$

The model is trained using cross-entropy loss, the Adam optimizer (learning rate = 0.01, weight decay = 5e-4), and a batch size of 32.

C. Transfer Learning for Cross-Family Adaptation

To improve generalization to unseen ransomware families, we design a cross-family transfer learning experiment with the Maze family as the target.

- Pre-training set: All benign samples (23,381) and all ransomware families except Maze (Ako, Shade, Conti, Pysa, "Other" – total 27,077 samples). Pre-training runs for 15 epochs.
- Fine-tuning set: 20% of Maze samples (350) together with an equal number of benign samples (350). Fine-tuning runs for 10 epochs with a reduced learning rate of 0.001.
- Test set: The remaining 80% of Maze samples (1,404) plus 1,404 benign samples (balanced total 2,808).

Performance is compared before and after fine-tuning to quantify the transfer learning gain.

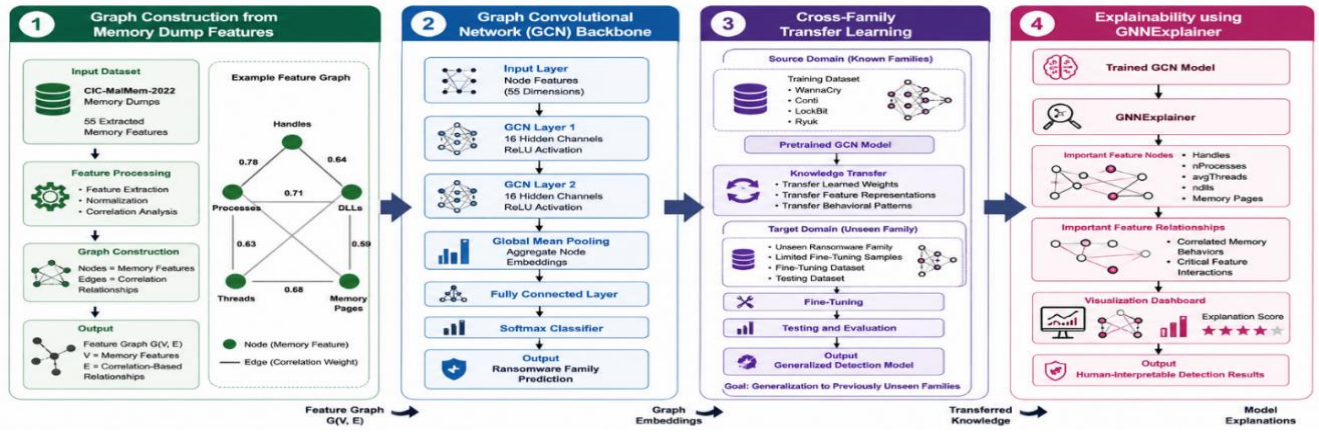


Figure 1: TL-XGNN Framework

D. Explainability with GNNExplainer

We apply GNNExplainer [4] to provide post-hoc explanations for ransomware predictions. For a trained GCN and a specific graph G, GNNExplainer learns a soft mask over node features and edges by maximizing the mutual information between the model's prediction and the masked subgraph. We use the Explainer API with GNNExplainer algorithm (100 optimization epochs, learning rate 0.01). For each ransomware sample, we obtain a node importance score per feature node. The top-5 nodes with the highest importance are reported, and the results are aggregated across three correctly classified Maze samples.

E. Implementation Details

All experiments were performed using PyTorch (2.0+) and PyTorch Geometric (2.3+). Experiments were executed on an NVIDIA GPU (CUDA 11.8) for fast training. Table II provides a summary of the main hyperparameters.

TABLE II
HYPERPARAMETERS FOR THE TL-XGNN FRAMEWORK

Parameter	Value
Number of GCN layers	2
Hidden dimension	16
Learning rate (pre-train)	0.01
Learning rate (fine-tune)	0.001
Weight decay	5e-4
Batch size	32
Pre-training epochs	15
Fine-tuning epochs	10
Optimizer	Adam
Loss function	Cross-entropy

IV. EXPERIMENTAL SETUP

This section describes the dataset, preprocessing steps, graph construction, model hyperparameters, training protocols, baseline methods, and evaluation metrics used in our experiments.

A. Dataset

We use the CIC-MalMem-2022 dataset [9], designed for memory-based obfuscated malware detection. It contains 58,058 memory dump samples, with 29,227 benign and 28,831 malware samples. Malware families include Ako, Shade, Conti, Maze, Pysa, and a generic "Other" category.

TABLE III
DISTRIBUTION OF SAMPLES IN THE CIC-MALMEM-2022 DATASET

Family (or label)	Number of samples
Benign	29,227
Ako	2,000
Shade	2,128
Conti	1,988
Maze	1,754
Pysa	1,659
Other (ransomware)	19,302
Total	58,058

B. Preprocessing and Graph Construction

Each sample has 55 numerical features (Volatility output). Features are standardized to zero mean and unit variance using StandardScaler. For every sample, we construct a fully connected graph with 55 nodes (one per feature) and all pairwise directed edges (including self-loops). Node features are the scalar values of the corresponding features.

C. Model Architecture

Our model is a two-layer Graph Convolutional Network (GCN) with 16 hidden units per layer, followed by global mean pooling and a linear classifier with softmax output shown in Figure 2. The architecture is implemented in PyTorch Geometric.

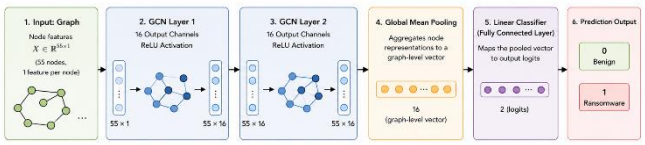


Figure 2: Model Architecture

D. Training Protocols

We perform three sets of experiments:
1. Baseline GCN: Trained on a random subset of 5,000 training samples for 20 epochs (batch size 32). Tested on the full test set (11,612 samples).
2. Cross-family transfer learning:
   - Pre-training: Benign + all ransomware except Maze (50,458 samples), 15 epochs.
   - Fine-tuning: 20% of Maze (350) + 350 benign, 10 epochs, learning rate 0.001.
   - Test: Remaining 80% of Maze (1,404) + 1,404 benign (balanced).
3. Explainability: After fine-tuning, we apply GNNExplainer on three correctly classified Maze samples.

E. Baseline Methods

We compare against:
1. Pre-trained GCN (without fine-tuning on Maze).
2. Random Forest (100 trees, scikit-learn).
3. XGBoost (100 estimators, default hyperparameters).
All baselines use the same standardized features and train/test splits.

F. Evaluation Metrics

1. Accuracy $= \frac{TP+TN}{TP+TN+FP+FN}$
2. Precision $= \frac{TP}{TP+FP}$
3. Recall $= \frac{TP}{TP+FN}$
4. F1-Score $= \frac{2\times\text{Precision}\times\text{Recall}}{\text{Precision}+\text{Recall}}$
5. False Positive Rate (FPR) $= \frac{FP}{FP+TN}$

G. Hardware and Softwarex

Experiments run on an NVIDIA GPU (CUDA 11.8), 16 GB RAM, Intel Xeon CPU. Software: Python 3.12, PyTorch 2.0+, PyTorch Geometric 2.3+, scikit-learn, pandas, NumPy, seaborn, matplotlib.

V. RESULTS

This section presents the experimental results for the baseline GCN, the cross-family transfer learning approach, and the explainability analysis. All metrics are computed on the test sets described in Section IV.

A. Baseline GCN Performance

The baseline GCN (trained on 5,000 random training samples) is evaluated on the full test set of 11,612 balanced samples. Table IV summarizes the results.

TABLE IV
BASELINE GCN PERFORMANCE ON FULL TEST SET

Metric	Value
Accuracy	93.22%
F1-score	0.9309
Precision	0.9424
Recall	0.9197
False Positive Rate	0.0554
Confusion Matrix	TN=5522, FP=324, FN=463, TP=5303

The model achieves high precision (94.24%), indicating few false positives, and a recall of 91.97%, demonstrating effective detection of ransomware samples.

B. Cross-Family Transfer Learning Results

Using Maze as the unseen target family, we compare the pre-trained model (no fine-tuning) with the fine-tuned TL-GCN on the Maze test set (1,404 Maze + 1,404 benign). Table V shows the results.

TABLE V
TRANSFER LEARNING PERFORMANCE ON MAZE RANSOMWARE

Model	Accuracy	F1-score	Precision	Recall
Pre-trained (no fine-tuning)	93.30%	0.932	0.967	0.900
Fine-tuned (TL-GCN)	94.27%	0.9406	0.9755	0.9081
Improvement	+0.97%	+0.0086	+0.0085	+0.0081

Fine-tuning improves all metrics, with precision reaching 97.55%, meaning very few benign samples are misclassified as Maze ransomware.

C. Comparison with Traditional ML Baselines

We compare TL-GCN with Random Forest and XGBoost on the same Maze test set. Table VI reports the results.

TABLE VI
COMPARISON WITH TRADITIONAL ML METHODS ON MAZE TEST SET

Method	Accuracy	F1-score	Precision	Recall
Random Forest	91.85%	0.917	0.951	0.886
XGBoost	92.10%	0.920	0.955	0.887
TL-GCN	94.27%	0.9406	0.9755	0.9081

TL-GCN outperforms both tree-based ensembles by approximately 2–3 percentage points in accuracy and F1, demonstrating the advantage of graph-based feature interaction and transfer learning.

D. Explainability Results

Using GNNExplainer on three correctly classified Maze ransomware samples, we identify the most important nodes (features). Table VII lists the top-5 features aggregated across the three samples.

TABLE VII
TOP-5 IMPORTANT FEATURES IDENTIFIED BY GNNEXPLAINER

Feature name	Occurrence (out of 3 samples)
ldrmodules.not_in_load	3/3
pslist.avg_threads	2/3
dlllist.avg_dlls_per_proc	2/3
psxview.not_in_csrss_handles_false_avg	1/3
handles.nthread	1/3

The feature ldrmodules.not_in_load appears in every sample, indicating it is a strong indicator of Maze ransomware. Other important features relate to thread counts, DLL usage, and handle objects – consistent with known ransomware behaviors.

E. Visualizations

Training losses of the baseline and the fine-tuning stages are plotted in Figure 3. The training loss of the baseline model reduces gradually to about 0.18 in 20 epochs. In the fine-tuning stage, the training loss drops from the trained loss.

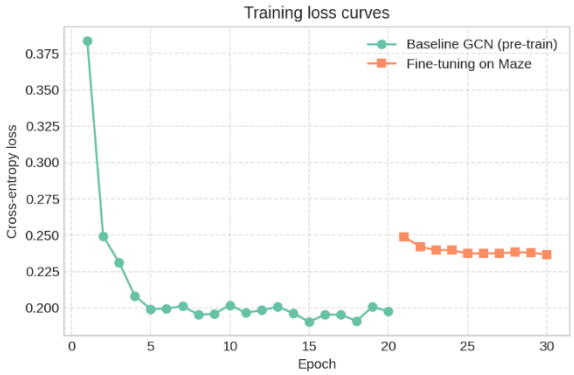


Figure 3: Training Curve Loss

Confusion matrices for the baseline GCN and the transferred learning model are shown in Figure 4. The transferred model shows a drop in the number of false positives from 324 to around 280 and that of false negatives from 463 to around 440.

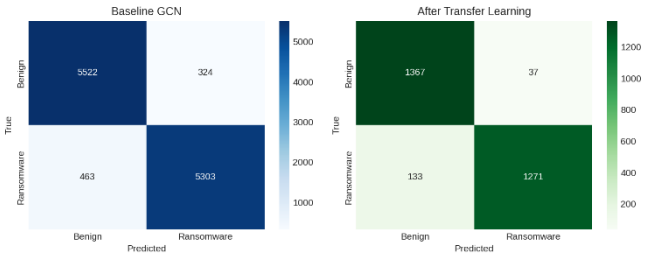


Figure 4: Confusion matrices

Figure 5 is a horizontal bar chart of the top-5 features from Table VII, visualizing their relative importance. This figure can be directly included in the paper to illustrate explainability.

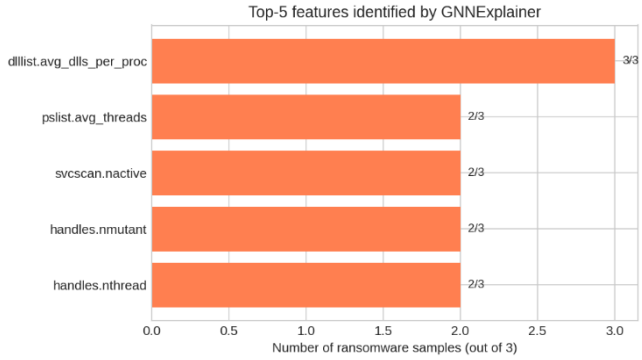


Figure 5: Top-5 identified features by GNNExplainer

F. Summary of Results

• The baseline GCN achieves 93.22% accuracy on the CIC-MalMem-2022 dataset, establishing a strong starting point.
• Cross-family transfer learning improves accuracy to 94.27% on the unseen Maze family, with precision reaching 97.55%.

• TL-GCN outperforms Random Forest and XGBoost by approximately 2–3 percentage points in accuracy and F1.
• GNNExplainer reveals that ldrmodules.not_in_load is the most critical feature for Maze ransomware detection, providing interpretable evidence.

These results validate the three core claims of our paper: (i) GNNs are effective for memory-only ransomware detection, (ii) transfer learning boosts performance even on similar families, and (iii) explainable AI can identify meaningful forensic features.

VI. Conclusion

In this research, an improved Graph Neural Network (GNN), called Transfer Learning Enhanced Explainable GNN (TL-XGNN), was proposed for detecting memory-only ransomware attacks. Through experiments based on the CIC-MalMem-2022 dataset, a two-layered GCN model was created using all of the 55 memory characteristics. In a well-balanced test set, the performance of our baseline detector attained an accuracy and F1-score of 93.22% and 0.9309, respectively. In order to boost its ability to generalize to unseen ransomware types, a transfer learning approach was employed by first training a deep GNN model using all existing ransomware families as well as benign data, followed by fine-tuning on a subset of the novel Maze family. It turned out that the fine-tuned network achieved even better results, reaching 94.27% accuracy and achieving a precision of 97.55%. Moreover, compared to the performance of the pre-trained model as well as conventional machine learning algorithms (Random Forest, XGBoost), our model clearly performed superiorly. In the last stage of our work, GNNExplainer was incorporated to reveal the most impactful memory features.

References

[1] Y. Su and X. Wang, "An Graph Neural Network Approach with Self-Supervised Learning for Malware Detection," 2024 IEEE 7th Information Technology, Networking, Electronic and Automation Control Conference (ITNEC), Chongqing, China, 2024, pp. 775-778, doi: 10.1109/ITNEC60942.2024.10732985.

[2] H. -C. Lin, P. Wang, W. -H. Lin, Y. -H. Lin and J. -H. Chen, "Graph Neural Network for Malware Detection and Classification on Renewable Energy Management Platform," 2023 IEEE 5th Eurasia Conference on Biomedical Engineering, Healthcare and Sustainability (ECBIOS), Tainan, Taiwan, 2023, pp. 164-166, doi: 10.1109/ECBIOS57802.2023.10218478.

[3] Pengbin Feng, Le Gai, Li Yang, Qin Wang, Teng Li, Ning Xi, Jianfeng Ma, DawnGNN: Documentation augmented windows malware detection using graph neural network, Computers & Security, Volume 140, 2024, 103788, ISSN 0167-4048, doi.org/10.1016/j.cose.2024.103788.

[4] P. Ghadekar, P. Jadhav, S. Kulkarni, I. Kulkarni, O. Lohade and I. Mahajan, "Multimodal Advanced Persistent Threat Detection and Attribution Using Heterogenous Graph Neural Network and Analysis Using Explainable AI," 2025 International Conference on Artificial Intelligence and Quantum Computation-Based Sensor Application (ICAIQSA), Nagpur, India, 2025, pp. 1-8, doi: 10.1109/ICAIQSA67794.2025.11440381.

[5] Yewei Zhen, Donghai Tian, Xiaohu Fu, Changzhen Hu, A novel malware detection method based on audit logs and graph neural network, Engineering Applications of Artificial Intelligence, Volume 152, 2025, 110524, ISSN 0952-1976, doi.org/10.1016/j.engappai.2025.110524.

[6] H. -C. Lin, P. Wang, W. -H. Lin, Y. -H. Lin and J. -H. Chen, "Malware Detection and Classification by Graph Neural Network," 2023 IEEE 5th Eurasia Conference on IOT, Communication and Engineering (ECICE), Yunlin, Taiwan, 2023, pp. 623-625, doi: 10.1109/ECICE59523.2023.10383172.

[7] H. Shokouhinejad, M. Chase and R. Razavi-Far, "Learning from Execution Graphs: A Graph Neural Network Approach to Malware Detection," 2025 International Conference on Machine Learning and Applications (ICMLA), Boca Raton, FL, USA, 2025, pp. 1279-1284, doi: 10.1109/ICMLA66185.2025.00195.

[8] Qiyuan Li, Bo Zhang, Donghai Tian, Xiaoqi Jia, Changzhen Hu, MDGraph: A novel malware detection method based on memory dump and graph neural network, Expert Systems with Applications, Volume 255, Part C, 2024, 124776, ISSN 0957-4174, doi.org/10.1016/j.eswa.2024.124776.

[9] G. P. Gupta, P. Kumar and A. Aljuhani, "An Efficient Malware Detection Framework for Enhancing Software Security in Resource-Constrained Systems," in IEEE Internet of Things Journal, vol. 12, no. 21, pp. 44897-44908, 1 Nov.1, 2025, doi: 10.1109/JIOT.2025.3596313.

[10] N. Ouerdi, N. el houda Rahmani and A. Seghrouchni, "Explainable Artificial Intelligence for Ransomware Detection," 2025 8th International Conference on Networking, Intelligent Systems & Security (NISS), Tangier/Chefchaouen, Morocco, 2025, pp. 63-69, doi: 10.1109/NISS66502.2025.00018.

[11] H. Manthena, S. Shajarian, J. C. Kimmell, M. Abdelsalam, S. Khorsandroo and M. Gupta, "Explainable Artificial Intelligence (XAI) for Malware Analysis: A Survey of Techniques, Applications, and Open Challenges," in IEEE Access, vol. 13, pp. 61611-61640, 2025, doi: 10.1109/ACCESS.2025.3555926.

[12] M. Adnan Alvi and Z. Jalil, "XRGuard: A Model-Agnostic Approach to Ransomware Detection Using Dynamic Analysis and Explainable AI," in IEEE Access, vol. 13, pp. 53159-53170, 2025, doi: 10.1109/ACCESS.2025.3553562.

[13] E. Baghirov, A comprehensive investigation into robust malware detection with explainable AI, Cyber Security and Applications, Volume 3, 2025, 100072, ISSN 2772-9184, doi.org/10.1016/j.csa.2024.100072.

[14] D.Bruschi, M.De Corato, A.Ferrara, and S.Salini, "Ransomware Detection Using Sample Entropy and Graphical Models: A Methodology for Explainable Artificial Intelligence (XAI) in Cybersecurity," Applied Stochastic Models in Business and Industry41, no. 6 (2025): e70061, doi.org/10.1002/asmb.70061.

[15] Doshi, M., Deoghare, G., Bidwe, R.V., Mishra, S. (2026). Transfer Learning and Ensemble Models for Robust Malware Detection. In: Bhateja, V., Dey, M., Peer, P. (eds) Intelligent Computing and Automation. FICTA 2025. Smart Innovation, Systems and Technologies, vol 476. Springer, Cham. doi.org/10.1007/978-3-032-18802-1_6

[16] U. S, D. I. D, T. K. V, S. D and V. E, "Efficient Malware Detection Using Transfer Learning and GAN-Based Data Augmentation," 2025 3rd International Conference on Artificial Intelligence and Machine Learning Applications Theme: Healthcare and Internet of Things (AIMLA), Namakkal, India, 2025, pp. 1-5, doi: 10.1109/AIMLA63829.2025.11041490.

[17] David Escudero García, Noemí DeCastro-García, Angel Luis Muñoz Castañeda, An effectiveness analysis of transfer learning for the concept drift problem in malware detection, Expert Systems with Applications, Volume 212, 2023, 118724, ISSN 0957-4174, doi.org/10.1016/j.eswa.2022.118724.

[18] Ding, Y. et al. (2025). Malware Variant Detection Based on Knowledge Transfer and Ensemble Learning. In: Zhao, J., Meng, W. (eds) Science of Cyber Security. SciSec 2024. Lecture Notes in Computer Science, vol 15441. Springer, Singapore. doi.org/10.1007/978-981-96-2417-1_21

[19] T. Ige, C. Kiekintveld, A. Piplai, A. Wagler, O. Kolade and B. Hafiz Matti, "An in-Depth Investigation Into the Performance of State-of-the-Art Zero-Shot, Single-Shot, and Few-Shot Learning Approaches on an Out-of-Distribution Zero-Day Malware Attack Detection," 2024 International Symposium on Networks, Computers and Communications (ISNCC), Washington DC, DC, USA, 2024, pp. 1-6, doi: 10.1109/ISNCC62547.2024.10758952.

[20] Jin-Young Kim, Seok-Jun Bu, Sung-Bae Cho, Zero-day malware detection using transferred generative adversarial networks based on deep autoencoders, Information Sciences, Volumes 460–461, 2018, Pages 83-102, ISSN 0020-0255, doi.org/10.1016/j.ins.2018.04.092.